

Despliegue en Kubernetes con Load Balancer y Conexión Híbrida AWS

Realizado por Rodrigo María Gómez

1. Introducción y Objetivos

En esta práctica he implementado una arquitectura de microservicios utilizando Kubernetes (versión ligera k3s) en un entorno local WSL2. El objetivo principal ha sido desplegar una aplicación web escalable, gestionada por un Deployment y expuesta mediante un Load Balancer. Posteriormente, he conectado este clúster local con una instancia EC2 en AWS mediante un túnel SSH inverso, demostrando la capacidad de exponer servicios locales a la nube de forma segura.

2. Preparación del Entorno Local (WSL2)

2.1 Instalación de k3s y kubectl

Para comenzar, preparé mi entorno en Ubuntu (WSL2). Realicé la instalación de **k3s** evitando el uso de Docker para minimizar el consumo de recursos, y configuré **kubectl** para la gestión del clúster.

Verifiqué que el servicio estaba activo y que el nodo master estaba listo para recibir cargas de trabajo.

```
rodrigo@A6Alumno11:/mnt/c/WINDOWS/system32$ sudo k3s --version
k3s version v1.33.6+k3s1 (b5847677)
go version go1.24.9

rodrigo@A6Alumno11:/mnt/c/WINDOWS/system32$ sudo k3s kubectl get nodes
NAME                STATUS    ROLES                  AGE      VERSION
a6alumno11          Ready    control-plane,master   4m42s    v1.33.6+k3s1
```

CAPTURA 1: En esta imagen se observa la versión instalada de k3s y, lo más importante, el estado Ready del nodo master. Esto valida que el clúster local se ha inicializado correctamente y está operativo para recibir despliegues.

3. Desarrollo de la Aplicación

3.1 Creación de los archivos fuente

Creé una estructura de directorio para alojar la aplicación. Desarrollé una aplicación en Python con **Flask** (app.py) diseñada específicamente para devolver información del Pod (nombre, namespace e IP) y un archivo index.html para visualizar el balanceo de carga en el navegador. También generé el archivo requirements.txt y un Dockerfile base para asegurar la portabilidad del entorno.

```
rodrigo@A6Alumno11:~/kubernetes-aws-practice$ ls -R
.:
app
./app:
Dockerfile app.py index.html requirements.txt
```

CAPTURA 2: Esta captura confirma la creación correcta de todos los archivos fuente necesarios (app.py, index.html, etc.) en el directorio de trabajo local antes de proceder a la configuración de los manifiestos de Kubernetes.

4. Configuración y Despliegue en Kubernetes

4.1 Definición de Objetos (Manifests)

En esta fase, procedí a "kubernetesar" la aplicación. Creé los siguientes manifiestos YAML:

1. **Namespace:** Para aislar los recursos bajo load-balancer-demo.
2. **ConfigMap:** Para inyectar el código de la aplicación directamente en los pods.
3. **Deployment:** Configuré 3 réplicas iniciales para garantizar alta disponibilidad.
4. **Service:** Definí un servicio de tipo LoadBalancer para distribuir el tráfico.

4.2 Verificación del Despliegue

Una vez aplicados los manifiestos, comprobé que los 3 pods estuvieran corriendo simultáneamente y que el servicio tuviera asignada su IP cluster.

```
rodrigo@A6Alumno11:~/kubernetes-aws-practice$ kubectl get all -n load-balancer-demo
```

NAME	READY	STATUS	RESTARTS	AGE
pod/web-app-6f9c9874d9-56xrw	1/1	Running	0	6m20s
pod/web-app-6f9c9874d9-r1jtl	1/1	Running	0	6m20s
pod/web-app-6f9c9874d9-zp2xv	1/1	Running	0	6m20s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/web-app-service	LoadBalancer	10.43.234.47	<pending>	80:32535/TCP	114s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/web-app	3/3	3	3	6m20s

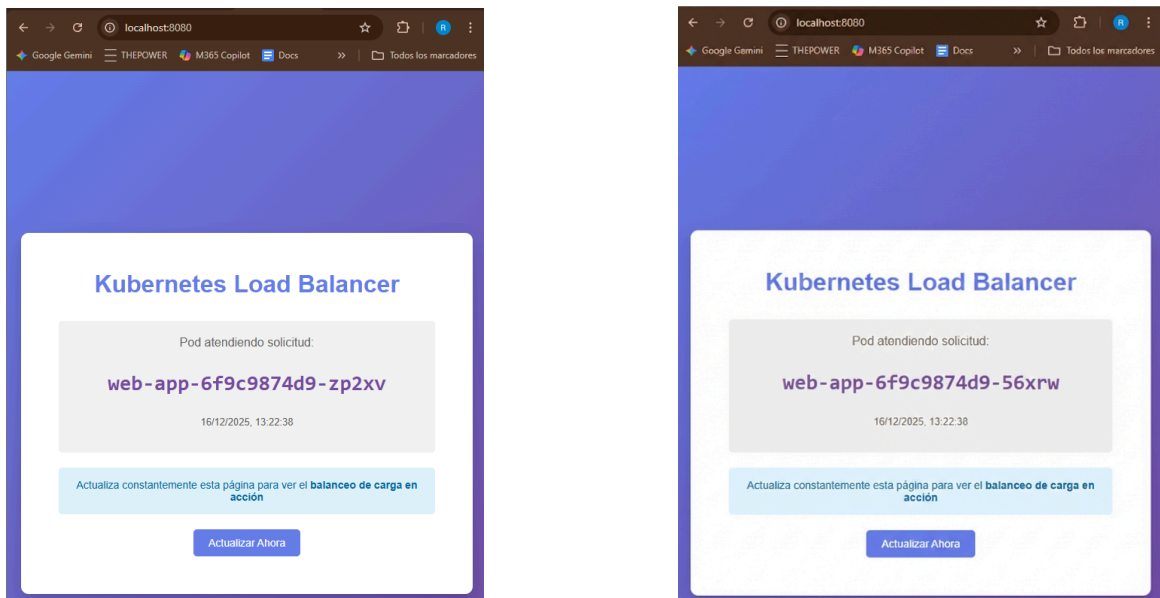
NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/web-app-6f9c9874d9	3	3	3	6m20s

CAPTURA 3: Aquí se verifica el despliegue exitoso de la arquitectura. Se pueden identificar los 3 pods en estado Running (garantizando alta disponibilidad), el servicio LoadBalancer escuchando en el puerto configurado y el Deployment controlando las réplicas.

5. Verificación Local del Balanceo de Carga

Para probar el acceso antes de conectar con la nube, utilicé port-forwarding para exponer el servicio en el puerto 8080 de mi máquina local.

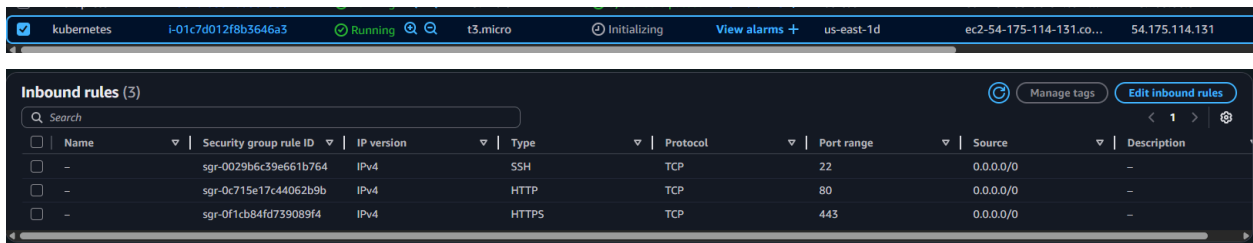
Accedí mediante el navegador y refresqué la página varias veces. Pude comprobar que el campo "Pod atendiendo solicitud" cambiaba, confirmando que el Load Balancer estaba distribuyendo el tráfico correctamente entre las réplicas (Round Robin).



CAPTURA 4: Esta captura demuestra la accesibilidad de la aplicación. Se visualiza la interfaz web servida desde dentro del clúster, y se identifica el nombre del pod específico que atendió esta petición en concreto, validando la inyección de variables de entorno.

6. Configuración de Infraestructura en AWS

6.1 Instancia EC2 y Seguridad



CAPTURA 5: Evidencia del aprovisionamiento de infraestructura en la nube. Se muestra la instancia EC2 en estado Running y se observa su dirección IP pública, dato indispensable para establecer posteriormente la conexión remota vía SSH. Creé un **Security Group** permitiendo tráfico SSH (22) y HTTP (80/443)

7. Conexión Híbrida: Kubernetes Local a AWS

7.1 Establecimiento del Túnel SSH

Este es el paso crítico de la práctica. Desde mi terminal local, establecí un **Túnel SSH Inverso (-R)**. Esto permitió redirigir el tráfico del puerto 8888 de la instancia AWS hacia mi servicio Kubernetes local (puerto 8080).

```
rodrigo@A6Alumno11:/mnt/c/WINDOWS/system32$ ssh -i ~/.ssh/kubernetes.pem -N -R 8888:localhost:8080 ubuntu@54.175.114.131
```

CAPTURA 6: Muestra la ejecución del comando clave para la conectividad híbrida. El flag -R configura el reenvío de puertos remoto, permitiendo que la instancia en la nube acceda a mi servicio local k3s a través del túnel seguro SSH, sin exponer mi red doméstica.

7.2 Prueba de Balanceo desde la Nube

Conectado a la instancia EC2, creé y ejecuté el script de prueba test-kubernetes-lb.sh. El script realizó múltiples peticiones internas a localhost:8888.

El resultado confirmó que, aunque la petición se originaba en la nube de AWS, estaba siendo atendida por mi clúster local, y el tráfico se balanceaba correctamente entre los distintos pods.

```
ubuntu@ip-172-31-24-135:~$ ~/test-kubernetes-lb.sh
=== Prueba Kubernetes Load Balancer desde AWS ===

Petición 1 -> Pod: web-app-6f9c9874d9-56xrw
Petición 2 -> Pod: web-app-6f9c9874d9-zp2xv
Petición 3 -> Pod: web-app-6f9c9874d9-rljtl
Petición 4 -> Pod: web-app-6f9c9874d9-56xrw
Petición 5 -> Pod: web-app-6f9c9874d9-zp2xv
Petición 6 -> Pod: web-app-6f9c9874d9-rljtl
Petición 7 -> Pod: web-app-6f9c9874d9-56xrw
Petición 8 -> Pod: web-app-6f9c9874d9-zp2xv
Petición 9 -> Pod: web-app-6f9c9874d9-rljtl
Petición 10 -> Pod: web-app-6f9c9874d9-56xrw
Petición 11 -> Pod: web-app-6f9c9874d9-zp2xv
Petición 12 -> Pod: web-app-6f9c9874d9-rljtl
Petición 13 -> Pod: web-app-6f9c9874d9-56xrw
Petición 14 -> Pod: web-app-6f9c9874d9-56xrw
Petición 15 -> Pod: web-app-6f9c9874d9-zp2xv
Petición 16 -> Pod: web-app-6f9c9874d9-56xrw
Petición 15 -> Pod: web-app-6f9c9874d9-56xrw

=== Resumen Balanceo ===
Pod web-app-6f9c9874d9-zp2xv: 5 peticiones
Pod web-app-6f9c9874d9-rljtl: 4 peticiones
Pod web-app-6f9c9874d9-56xrw: 6 peticiones
```

CAPTURA 7: Esta es la validación final de la arquitectura. El script, ejecutándose en AWS, recibe respuestas de diferentes pods locales. Esto confirma que el balanceo de carga está funcionando y atravesando correctamente el túnel SSH.

8. Escalado y Observabilidad

8.1 Escalado Dinámico

Para probar la elasticidad de Kubernetes, escalé el deployment de 3 a 5 réplicas en caliente utilizando el comando `kubectl scale`.

Verifiqué que los nuevos pods se inicializaran correctamente.

```
rodrigo@A6Alumno11:/mnt/c/WINDOWS/system32$ kubectl get pods -n load-balancer-demo
NAME                                READY   STATUS    RESTARTS   AGE
web-app-6f9c9874d9-56xrw            1/1     Running   1 (41m ago)  19h
web-app-6f9c9874d9-659c6            1/1     Running   0           40m
web-app-6f9c9874d9-8s9h2            1/1     Running   0           40m
web-app-6f9c9874d9-rljtl            1/1     Running   1 (41m ago)  19h
web-app-6f9c9874d9-zp2xv            1/1     Running   1 (41m ago)  19h
```

CAPTURA 8: Demostración de la elasticidad de Kubernetes. Tras ejecutar el comando de escalado, se observa que el número de pods ha aumentado de 3 a 5 y todos han alcanzado el estado Running o ContainerCreating, listos para asumir mayor carga de trabajo.

8.2 Monitoreo de Recursos

Finalmente, utilicé las herramientas de métricas de Kubernetes para observar el consumo de CPU y memoria de los pods tras la carga de trabajo.

```
ubuntu@ip-172-31-24-135:~$ ~/test-kubernetes-lb.sh
=== Prueba Kubernetes Load Balancer desde AWS ===

Petición 1 -> Pod: web-app-6f9c9874d9-56xrw
Petición 2 -> Pod: web-app-6f9c9874d9-zp2xv
Petición 3 -> Pod: web-app-6f9c9874d9-659c6
Petición 4 -> Pod: web-app-6f9c9874d9-56xrw
Petición 5 -> Pod: web-app-6f9c9874d9-56xrw
Petición 6 -> Pod: web-app-6f9c9874d9-659c6
Petición 7 -> Pod: web-app-6f9c9874d9-rljtl
Petición 8 -> Pod: web-app-6f9c9874d9-8s9h2
Petición 9 -> Pod: web-app-6f9c9874d9-659c6
Petición 10 -> Pod: web-app-6f9c9874d9-zp2xv
Petición 11 -> Pod: web-app-6f9c9874d9-659c6
Petición 12 -> Pod: web-app-6f9c9874d9-8s9h2
Petición 13 -> Pod: web-app-6f9c9874d9-659c6
Petición 14 -> Pod: web-app-6f9c9874d9-zp2xv
Petición 15 -> Pod: web-app-6f9c9874d9-659c6

=== Resumen Balanceo ===
Pod web-app-6f9c9874d9-zp2xv: 3 peticiones
Pod web-app-6f9c9874d9-8s9h2: 2 peticiones
Pod web-app-6f9c9874d9-rljtl: 1 peticiones
Pod web-app-6f9c9874d9-56xrw: 3 peticiones
Pod web-app-6f9c9874d9-659c6: 6 peticiones
```

```
rodrigo@A6Alumno11:~$ kubectl top pods -n load-balancer-demo
NAME                                CPU(cores)   MEMORY(bytes)
web-app-6f9c9874d9-56xrw            2m           28Mi
web-app-6f9c9874d9-659c6            2m           24Mi
web-app-6f9c9874d9-8s9h2            2m           24Mi
web-app-6f9c9874d9-rljtl            1m           28Mi
web-app-6f9c9874d9-zp2xv            1m           29Mi
```

CAPTURA 9: Uso de herramientas de observabilidad (`kubectl top`). Se visualiza el consumo de recursos (CPU y Memoria) en tiempo real de cada pod, permitiendo monitorizar el rendimiento y detectar posibles cuellos de botella en la aplicación.

9. Conclusión

Se ha demostrado la capacidad de Kubernetes para orquestar contenedores, gestionar el balanceo de carga y escalar horizontalmente. Además, se ha comprobado la viabilidad de conectar entornos locales con la nube pública mediante túneles seguros, permitiendo escenarios de desarrollo y pruebas híbridos sin necesidad de exponer IPs públicas locales.

Se procedió a la eliminación de los recursos en AWS (terminación de instancia) y limpieza del namespace local para finalizar.